UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad decana de América

Facultad de Ingeniería de Sistemas e Informática

Escuela Profesional de Ingeniería de Sistemas.

Estructura de Datos

Prof. Gilberto A. Salinas

Semana 04
Listas Enlazadas simples y circulares

Ciclo 2026-1

Agenda

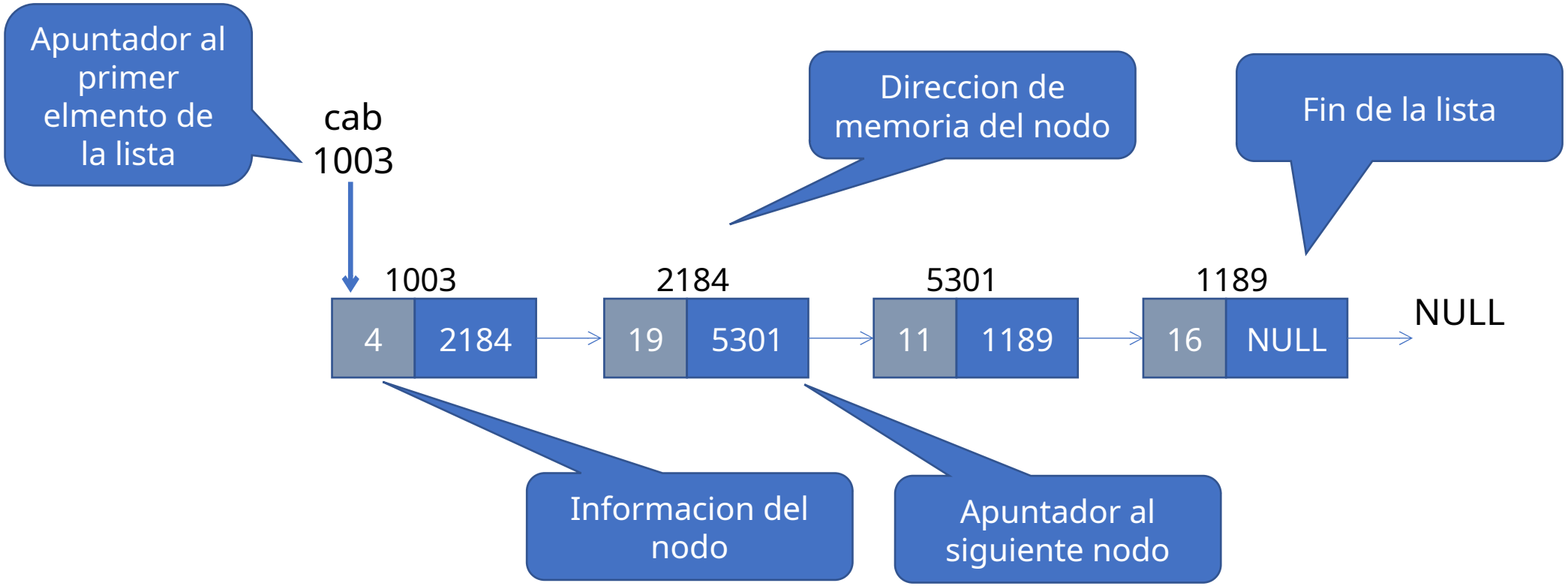
1. Introducción
2. Definición
   - Características
   - Ventajas/Desventajas
3. Representación/Declaracion
   - Lógica
   - Física
4. Tipos
5. Operaciones básicas
   - Análisis, diseño e implementación
6. Ejemplos
7. Resumen
8. Referencias

1. Listas enlazadas

1. **Introducción** . Son estructuras de datos fundamentales usadas para implementar incluso otras estructuras de datos.
   - Fueron desarrolladas por Cliff Shaw y Herbert Simon en 1956
   - Los lenguajes imperativos u orientados a objetos como C, C++, Java, Python, … disponen de referencias para crear listas enlazadas.

2. Definición

1. **Definición** . Es una estructura de datos lineal dinámica de un conjunto de datos homogéneos ligados entre si por una referencia al siguiente elemento, conformada de nodos donde cada nodo contiene la información que puede almacenar y un apuntador al siguiente nodo.



2. Definición

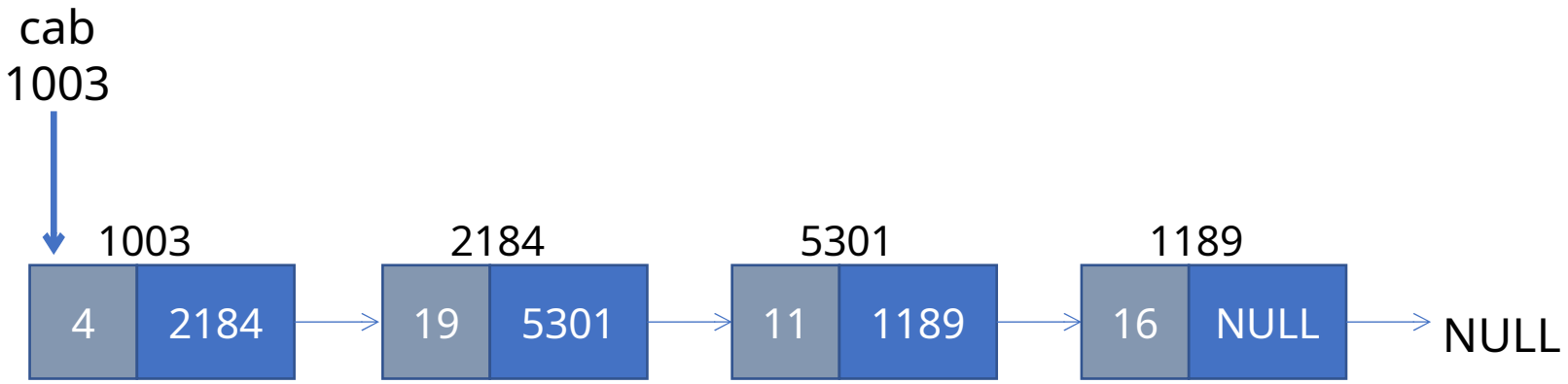
1. **Características**.
   - Son estructuras dinámicas que se crean y se eliminan en tiempo de ejecución
   - Pueden expandirse y contraerse fácilmente (inserciones y eliminaciones)
   - Se almacena en posiciones de memoria no contiguas.
   - El acceso a los elementos es secuencial.

2. Ventajas y Desventajas

1. **Ventajas**.
   - Son dinámicas, no se necesita saber el número de elementos a almacenar.
   - Son estructuras flexibles
   - Pueden expandirse y contraerse fácilmente
2. **Desventajas**.
   - Por su acceso secuencial no son buenas para búsquedas en grandes cantidades de datos.

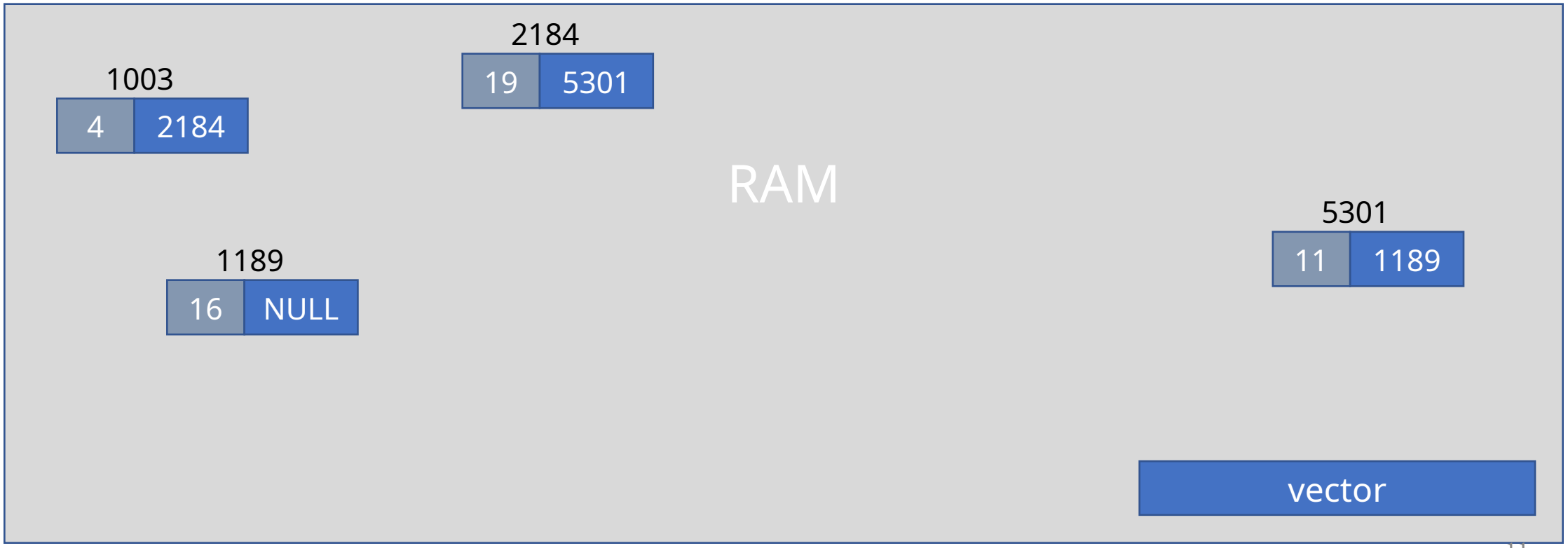
3. Representacion

- **Representación.** Se refiere a la representación lógica y física o como se representa en memoria.
  - **Lógica.** Traduce el esquema de datos conceptual en un esquema de datos lógico para la lista. Facilita la estructuración de datos



3. Representación

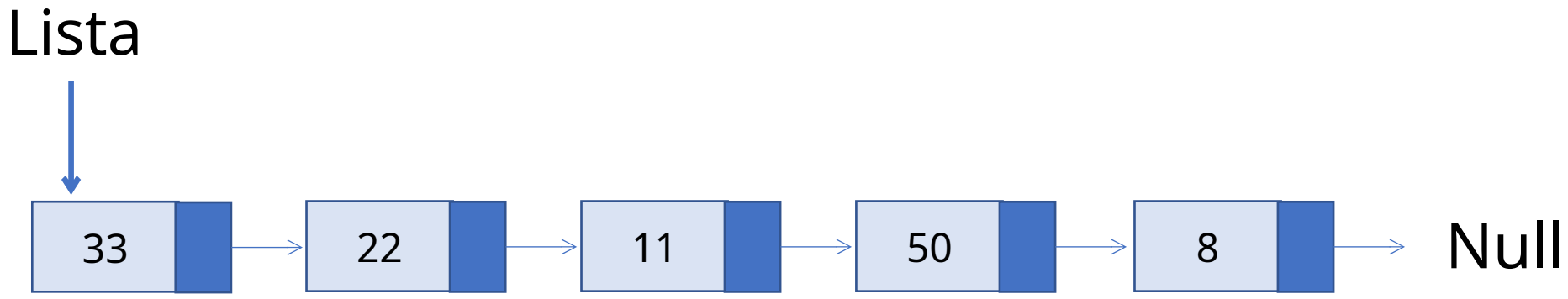
- **Representación.** Se refiere a la representación lógica y física o como se representa en memoria.
  - **Física.** Es la representación del esquema lógico en un esquema físico donde subyace la estructura **(RAM)** .



4. listas enlazadas

1. **Lista enlazada**.

- Es una estructura de datos en la que cada elemento apunta al siguiente así teniendo la referencia del inicio de la lista podemos acceder a todos los elementos de la misma.



4. Listas enlazadas: implementación

1. Metodología OO

- Leer detenidamente los requisitos funcionales de la lista.
- Análisis y diseño de la estructura de datos: ¿Que relaciones tendrán las clases de objetos o de clase?. Requiere trabajo de escritorio: Disene y especifique *Clases Nodo, ListaEnlazada y TListaEnlazada*: diagrama de clases de diseño debidamente relación y su cardinalidad.

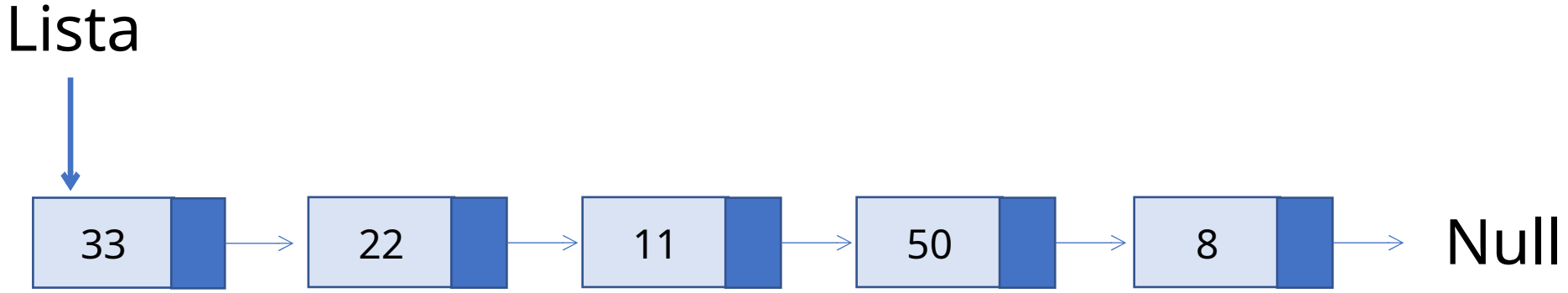
2. Metodología estructurada y modular

- Debemos primero la declarar estructura del NODO, luego las operaciones básicas en modulos.
- Requiere trabajo de escritorio: Analisis diseño y especficiacion.

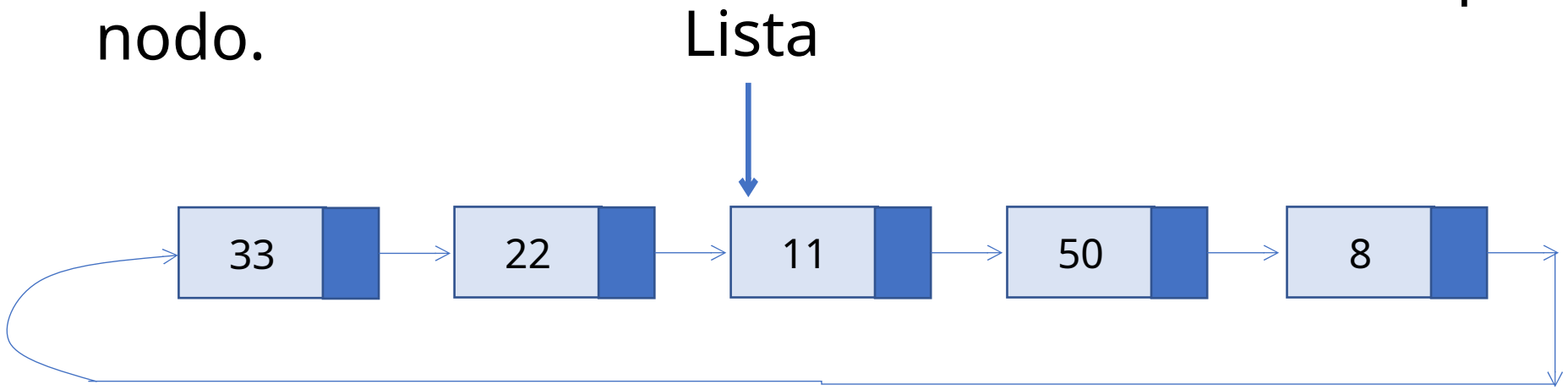
ACCION nombreOperacion (Lista cab, TD dato)
//Funcionalidad de la operación
PRE: requisitos de los datos de entrada
POS: requisitos de los datos de salida o solución

4. Tipos de listas enlazadas

1. **Listas Simples**. Son listas que tiene un único campo de enlace.

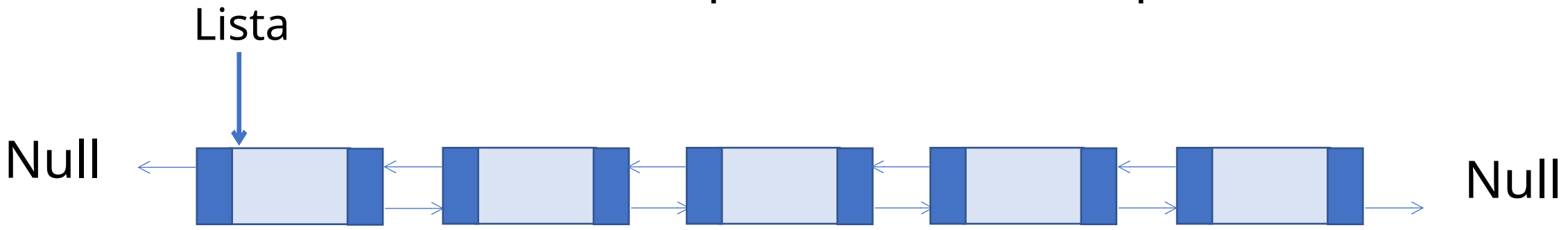


2. **Listas simples circulares**.
   - Es similar a la anterior donde el ultimo nodo apunta al primer nodo.

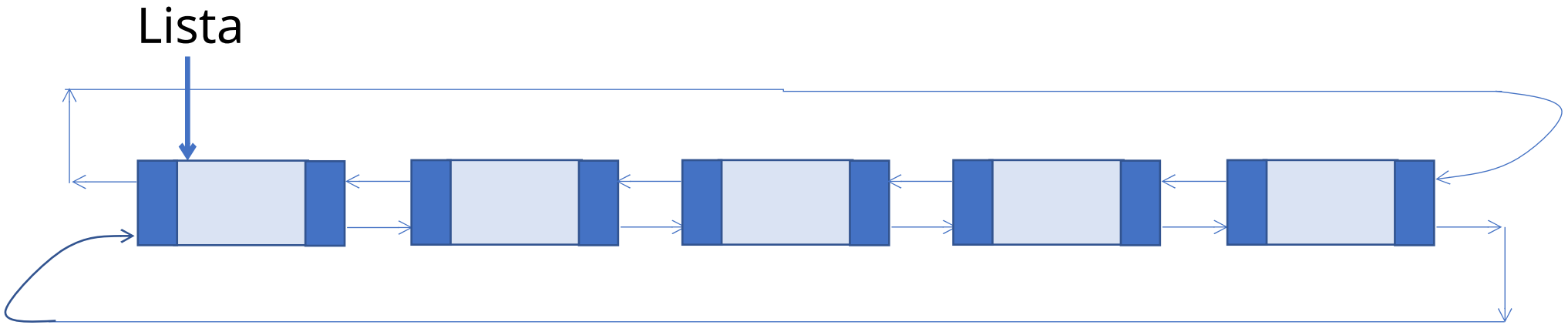


4. Tipos de listas enlazadas

1. **Listas Dobles**. Son listas que tiene dos campos de enlace.



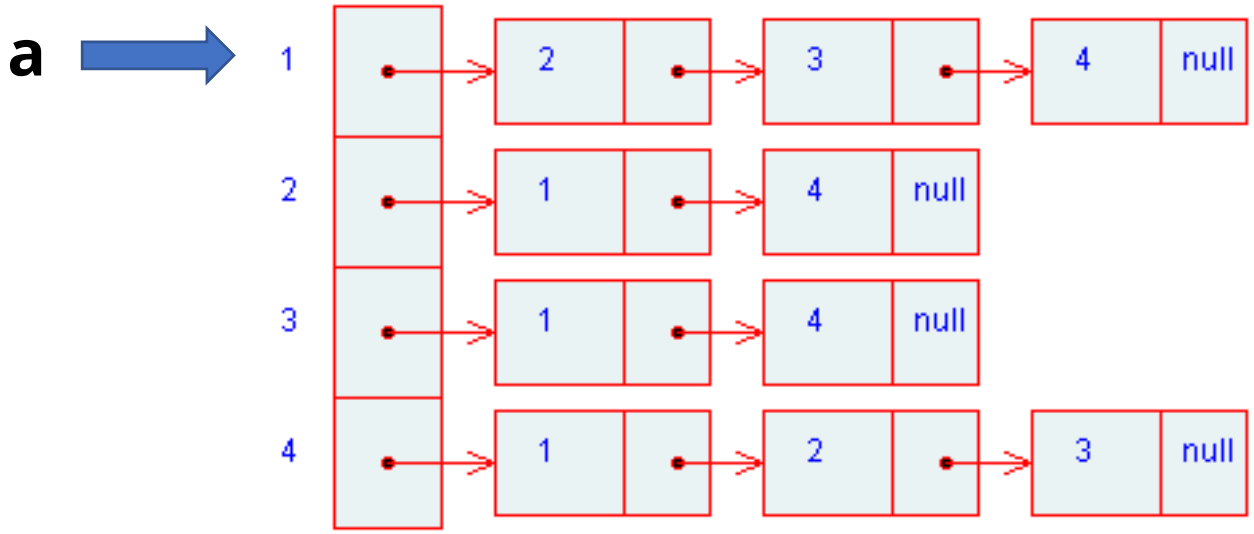
2. Listas dobles circulares.Son similares a la anterior donde el ultimo apunta al primero y el primer nodo apunta al ultimo nodo.



4. Tipos de listas enlazadas

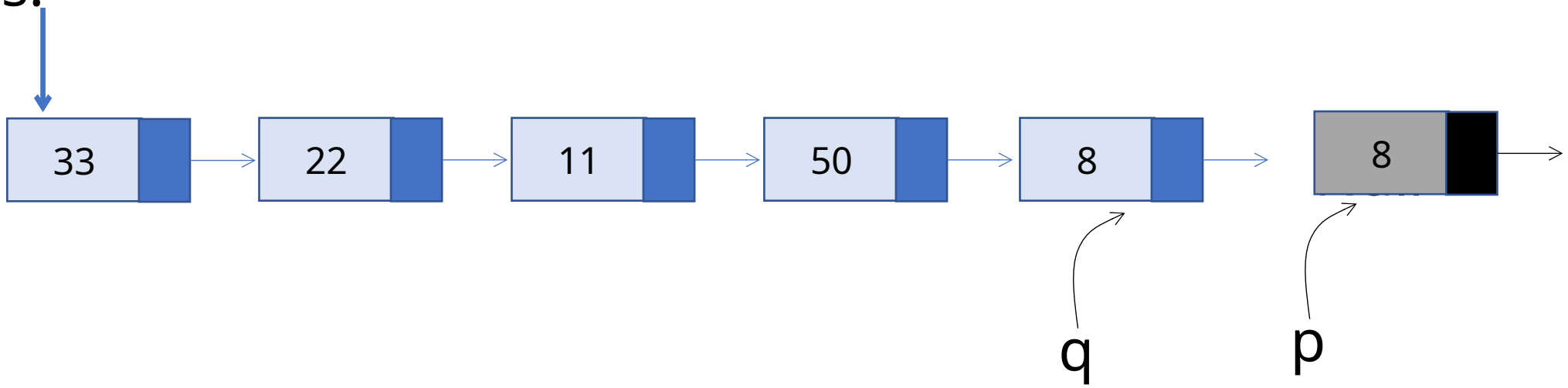
1. Listas especiales.

- Son listas que pueden tener mas de dos enlaces y permiten crear estructuras flexibles.
  - Árboles en general, arboles, grafos, …
  - Combinaciones de lista dobles con listas simples, etc.



5. Operaciones básicas: declaracion

1. **Listas enlazadas.** Las listas enlazadas están conformadas de nodos.




```
REGISTRO NODO
    TD  valor
    NODO *sgte
FIN_REGISTRO

NODO *lista, *p, *q
```

```
struct NODO{
    TD  valor;
    NODO *sgte;
}

NODO *lista, *p, *q;
```

```
public class Nodo<TD>{
    TD  valor;
    NODO *sgte;
    public Nodo() {}

    //setters
    //getters
}
```

```
public class Lista<TD>{
    NODO<TD> lista;
    public Lista() {}

    //setters
    //getters
}
```

5. Operaciones básicas: **Declaracion**

1. **Listas enlazadas.** Las listas enlazadas estan conformadas de nodos. Luego requiere crear las estructuras con las funcionalidades adecuadas.

```
REGISTRO  NODO
    TD   valor
    NODO sgte
FIN_REGISTRO
```

```
struct  NODO {
    TD  valor;
    NODO *sgte;
};
```

```
public Class  Nodo{
    Object  valor;
    Nodo sgte;
    public Nodo() {}
}
```

```
NODO  cab, p, q
```

```
NODO *cab, *p, *q;
```

```
Nodo cab, p, q;
```

5. Operaciones básicas

- **Constructoras**
  - Crear lista
  - …
- **Destructoras**
  - Eliminar lista
- **Modificadoras**
  - Adicionar un nodo a la lista.
  - Insertar nodo al inicio
  - Insertar un nodo antes o después de un nodo dato
  - …
- **Analizadoras**
  - Mostrar o visualizar el elementos de la lista
  - Buscar un determinado nodo
  - …
- **Persistencia**
  - Salvar y
  - Recuperar

6. Ejemplo

- Crear una lista enlazada simple de enteros.

Solución:

- **Definición Tipo ENTERO TD**
- **REGISTRO nodo**
- TD valor
- ENTERO sgte
- **FIN_REGISTRO**

6. Ejemplo

- Crear una lista enlazada doble de enteros.

Solución:

- **Definición Tipo ENTERO TD**
- **REGISTRO nodod**
- TD valor
- NODO sgte
  - NODO ante
- **FIN_REGISTRO**

6. Ejemplo

Diseno y especificacion de la opercion:

- **ACCION crearLista (Lista x)**
- //Crea una lista vacia
- PRE: Verdad
- POS: crea una referencia a lista vacia cab.

- **ACCION creatLista (Lista x)**
  - x = NULL
- **FIN_ACCION**

6. Ejemplo

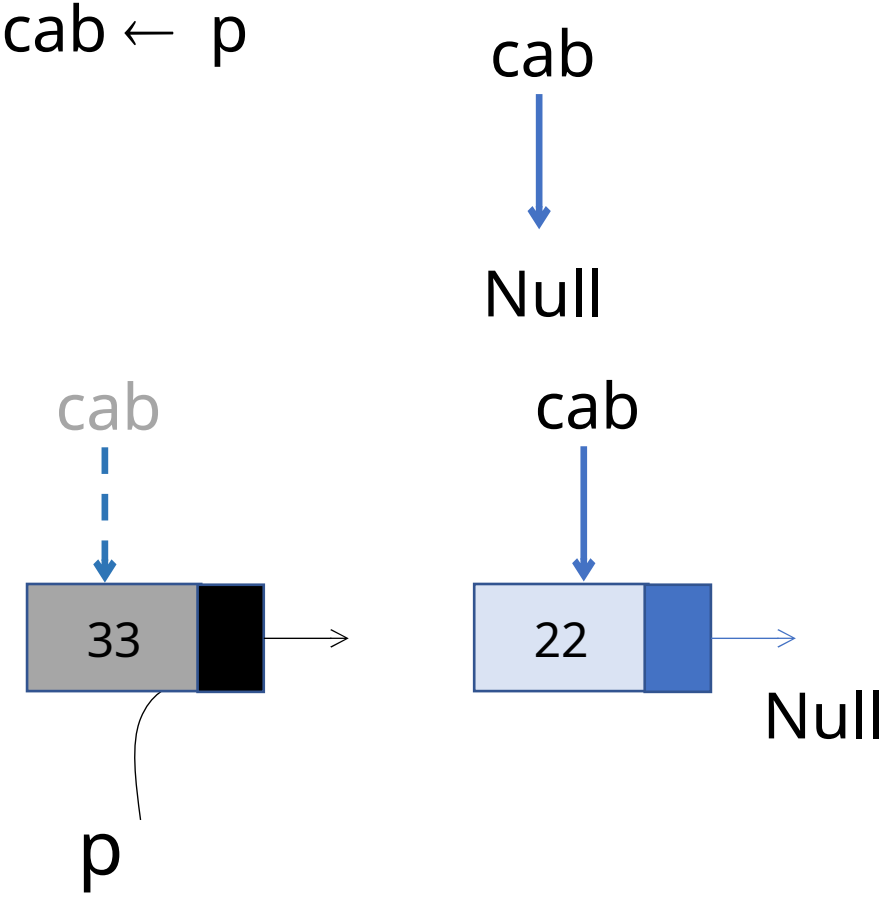
```
ACCION adicionaNodo (Lista x, TD dato,  Lista cab1)
    NODO p, q
    p ← nuevoNodo(cab1, dato)
    SI(p=NULL)
        ESCRIBIR("No hay espacio .....")
    SINO
        SI(cab=NULL)
            p.sgte ← cab
            Cab ← p
        SINO
            q ← cab
            MIENTRAS(q.sgte=NULL)
                q ← q.sgte
            FIN_MIENTRAS
            q.sgte = p
        FIN_SI
    FIN_SI
FIN_ACCION
```

6. Ejemplo

Inserta Inicio

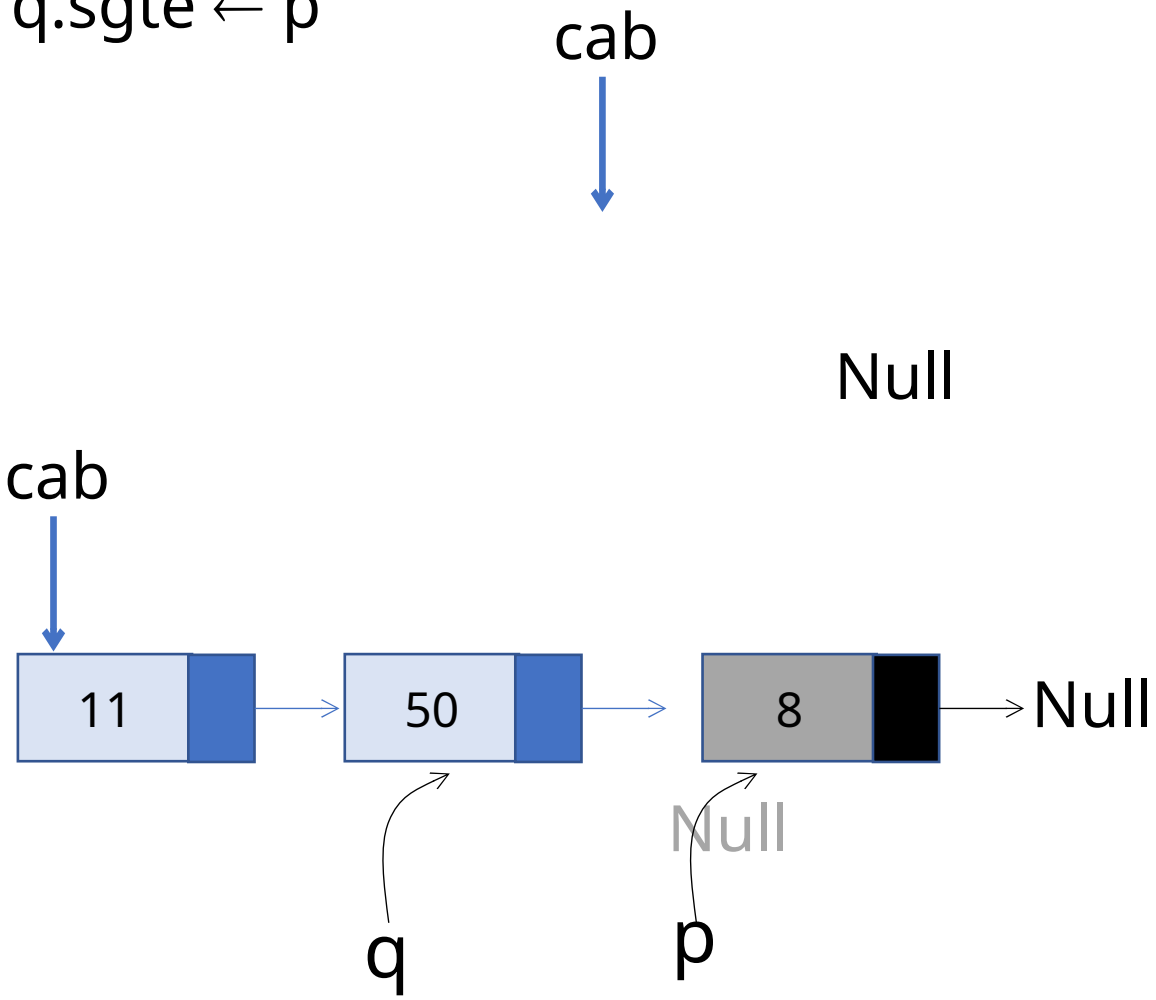
p.sgte ← cab

cab ← p



Inserta final

q.sgte ← p

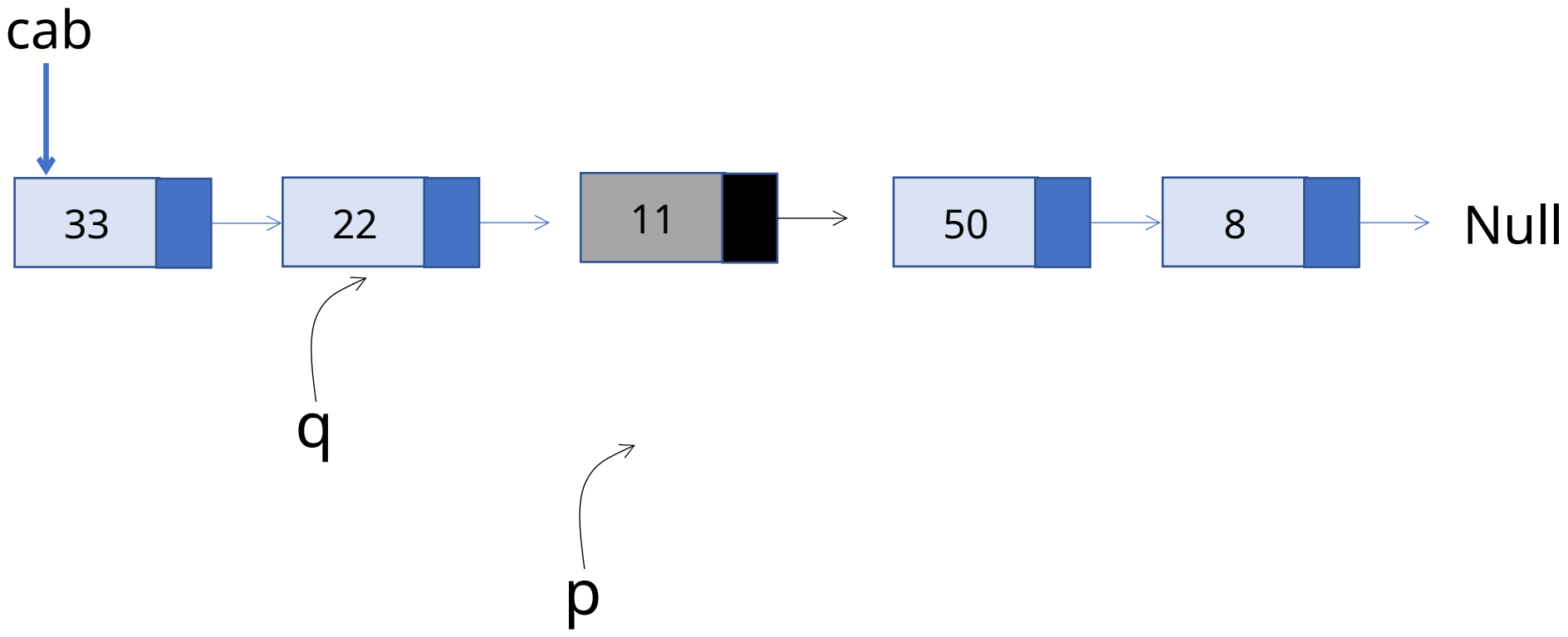


6. Ejemplo

Inserta nodo

p.sgte ← q.sgte

q.sgte ← p



6. Ejemplo

Elimina Inicio

p ← cab

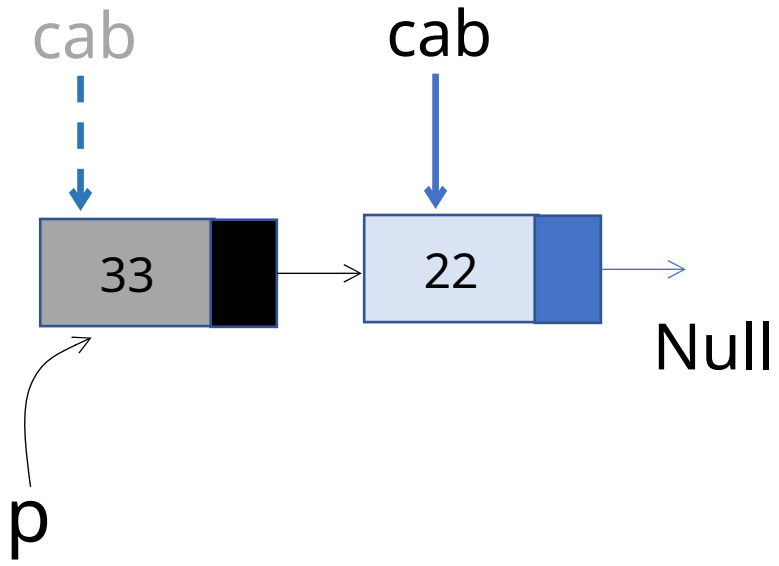
cab ← cab.sgte

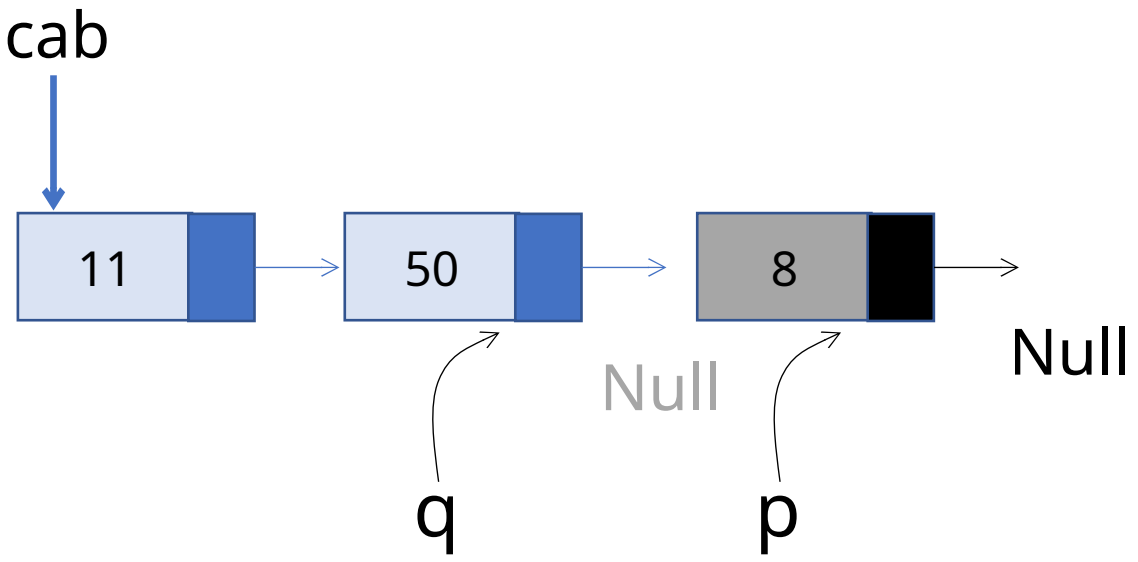
Libera(p)

Elimina final

q.sgte ← Null

Libera(p)



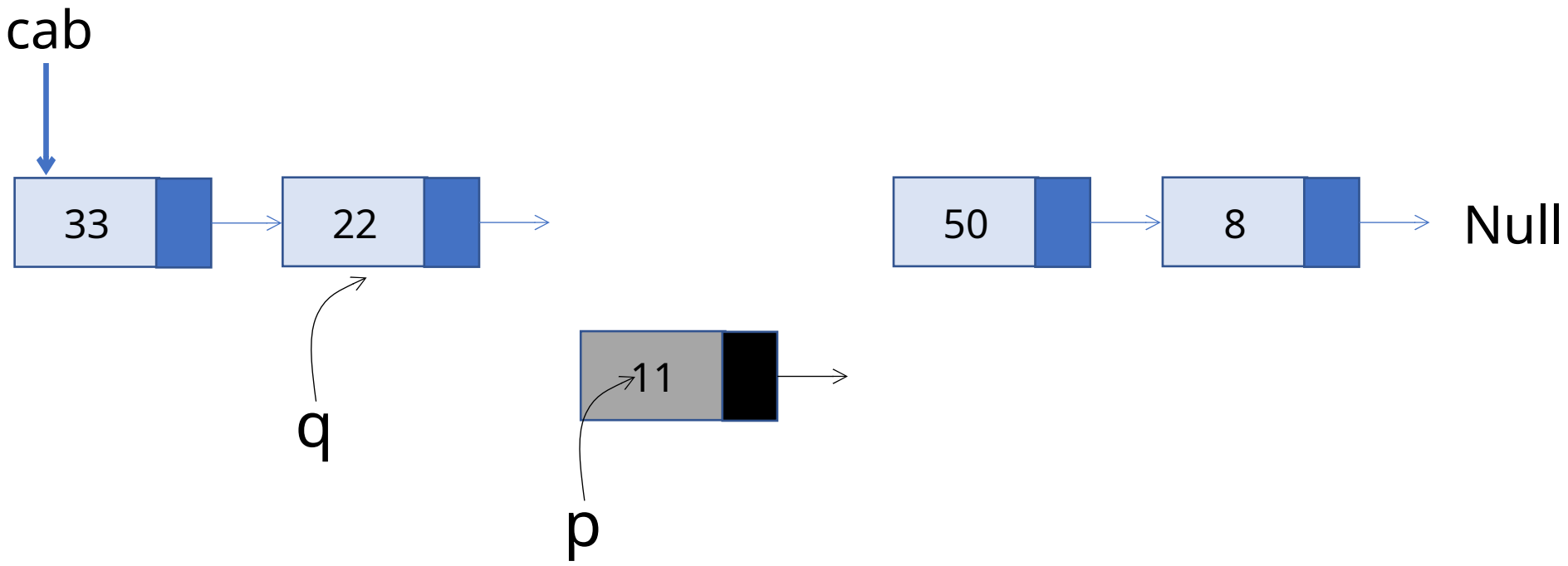


6. Ejemplo

Elimina nodo

q.sgte ← p.sgte

Libera(p)



6. Ejemplo

ACCION nuevoNodo(NODO cab1, TD dato)

```
    NODO p,
    P ← cab1
    SI(p =NULL)
        ESCRIBIR("No hay nodos
        libre...")
        RETORNAR(p)
    SINO
        p.valor ← dato
        p.sgte ← NULL
        cab1 ← cab1.sgte
        RETORNAR(p)
    FIN_SI
```

FIN_ACCION

ACCION liberaNodo(NODO cab1, NODO np)

```
    np.sgte ← cab1
    cab1 ← np
FIN_ACCION
```

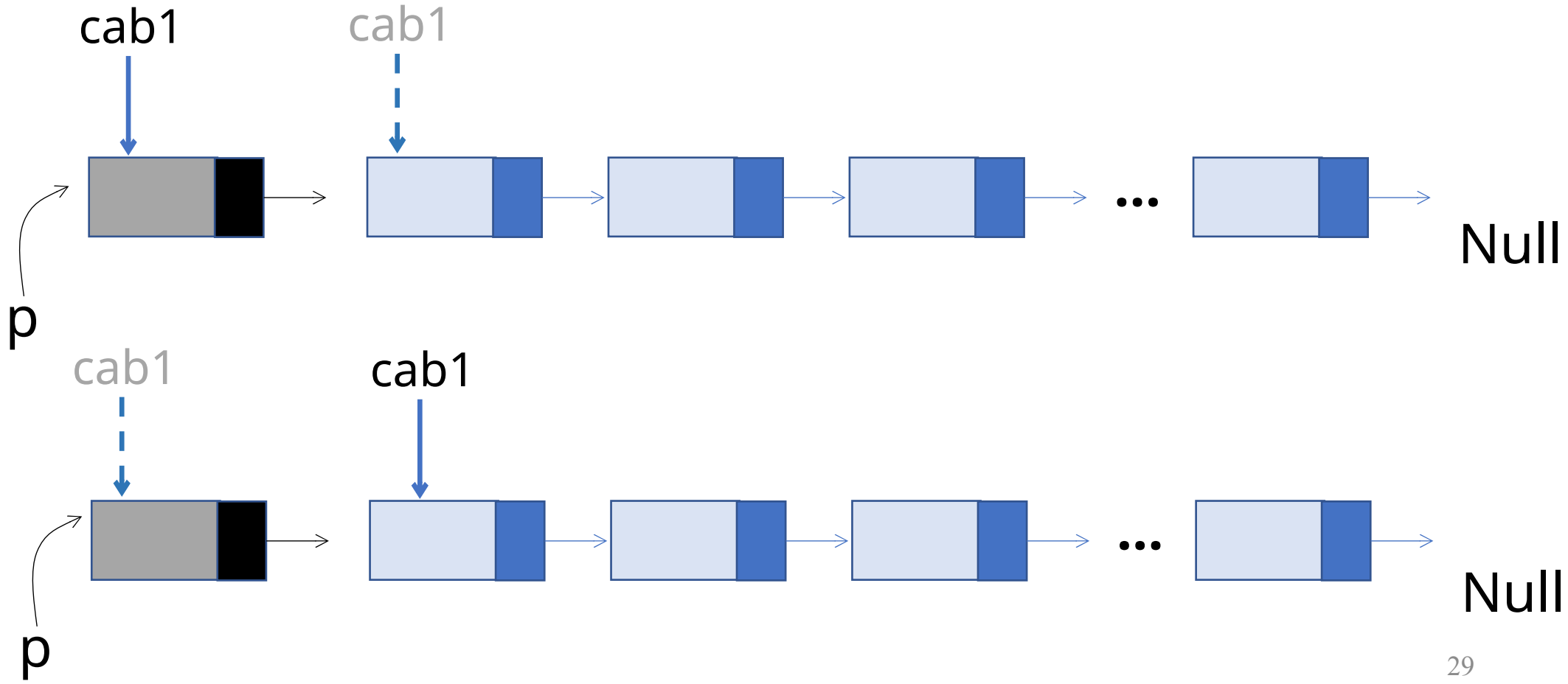
6. Ejemplo

p ← nuevoNodo(cab1, TD dato)

libera(p)

p ← (NODO*)malloc(sizeof(NODO));

free(p);



6. Ejemplo diseño y especificacion

ACCION buscarDato(NODO cab, TD dato)

// Busca el elemento dato en la lista referenciada por cab

PRE: cab=<> $\vee$ cab =<e1,e2,...,en> $\wedge$ dato $\in$cab

POS: “Lista vacia” $\vee$ dato $\in$ cab $\vee$ dato $\notin$ cab

ACCION eliminarDato(NODO cab, TD dato)

// Elimina el elemento dato de la lista referenciada por cab

PRE: cab=<> $\vee$ cab =<e1,e2,...,en> $\wedge$ dato $\in$cab

POS: “Lista vacia” $\vee$ dato $\in$ cab, dato se elimna $\vee$ dato $\notin$ cab, dato no se elimina

5. Resumen

- Las listas enlazadas son estructuras fundamentales para crear otras estructuras
- No se necesita saber cuantos elementos se van almacenar.
- Son muy eficientes donde continuamente se insertan o se eliminan elementos
- Son estructuras muy versátiles, puedes representar listas, arboles, diccionarios, grafos, pilas, colas, etc.
- Tener presente con esa versatilidad también viene con cierto grado de complejidad para el principiante.
- Siempre es posible implementar estructuras arbitrariamente complejas con listas enlazadas.

Referencias

- Joyanes L. y Zahonero I. (2004) Algoritmos y estructura de datos. Una perspectiva en C. Madrid España: McGraw-Hill.
- Cortez A. (2013) Algoritmica técnicas algorítmicas. Lima. Peru: CEPREDIN
- Joyanes L. y Zahonero I. (2008) Estructura de datos en Java. Madrid, España: McGraw-Hill.
- Cairo O y Guardati S. (2006) Estructura de datos. Mexico: McHraw-Hill.
- Villalobos J. (1996) Diseño y Manejo de Estructuras de Datos en C. Santa Fe de Bogota, Colombia: McGraw-Hill.
- Ficheros en C y C++ .[Ultima actualización 13.06.2017] [Accesado en Lima, 19.05.2020, 1820 horas], http://c.conclase.net/ficheros/